Hash Tables

Search with a constant time

Mohammed Said BELAID
said.belaid@univ-usto.dz

14/10/2026



Outline



Finding a student

ID	Name	Grade
1023	Ahmed	14
1045	Mohammed	16
1087	Lina	11
1120	Sara	15

Question

How can we quickly find the grade of student 1087?



Finding a student

ID	Name	Grade
1023	Ahmed	14
1045	Mohammed	16
1087	Lina	11
1120	Sara	15

Question

How can we quickly find the grade of student 1087? What if the list contains 100000 students?



Naive solution: Linear search

```
for (int i = 0; i < students.length; i++) {
    if (students[i].id == targetId) {
        return students[i].grade;
    }
}
```

- Check entries one by one.
- In the worst case, inspect every entry.
- Time complexity: $O(n)$.



Better solution: Binary search

- If entries are sorted by ID, compare with the middle entry.
- Discard half of the remaining entries at each step.
- Time complexity: $O(\log n)$.
- For $n = 1000000 \rightarrow lg_2(n) \approx 20$



Can we do better?

- We want to look up a value using its key.



Can we do better?

- We want to look up a value using its key.
- We would like the lookup to be fast.



Can we do better?

- We want to look up a value using its key.
- We would like the lookup to be fast.
- Could we use the key to tell us where to look?



Use the Key as an Index

```
Students[1023] = New Student("AHMED", 14);
Students[1045] = New Student("MOHAMED", 16);
Students[1087] = New Student("LINA", 11);
Students[1120] = New Student("SARA", 15);

// .......

System.out.println(Students[1087]);
```

- No search is needed.
- Array access by index takes $O(1)$ time.



The problem with Direct access

Example

Suppose IDs are:

- Big number: 8 700 087
- UUID: `5759456d-0ad2-4a3b-ba06-5bcac4f33481`
- Passport ID: `P<DZABENMOHAMED«MOHAMED 1234567891DZA8505204M2810243«<28`
- String: `"Name"`
- Any object type!

Problems

- A huge array may be required.
- Most positions may remain unused.
- Keys may not be integers.



The Idea: calculate an index

Instead of using the key directly, calculate an array index from it.

Key	$\rightarrow$	Function $\rightarrow$ Index
1087	$\rightarrow$	$f(1087) \rightarrow 7$



The Idea: calculate an index

Instead of using the key directly, calculate an array index from it.

Key	$\rightarrow$	Function $\rightarrow$ Index
1087	$\rightarrow$	$f(1087) \rightarrow 7$

Can we map many possible keys to a small array?



A first Hash function

Suppose the array has 100 positions, indexed from 0 to 99.
We can use:

$$h(k) = k \bmod 10$$

Key	Calculation	Index
1023	1023 mod 100	23
1045	1045 mod 100	45
1087	1087 mod 100	87
1120	1120 mod 100	20



A first Hash function

Suppose the array has 100 positions, indexed from 0 to 99.
We can use:

$$h(k) = k \bmod 10$$

Key	Calculation	Index
1023	1023 mod 100	23
1045	1045 mod 100	45
1087	1087 mod 100	87
1120	1120 mod 100	20

Can we map many possible keys to a small array?



A first Hash function

Suppose the array has 100 positions, indexed from 0 to 99.
We can use:

$$h(k) = k \bmod 10$$

Key	Calculation	Index
1023	1023 mod 100	23
1045	1045 mod 100	45
1087	1087 mod 100	87
1120	1120 mod 100	20

Can we map many possible keys to a small array? What would be the issue?



A first Hash function

Suppose the array has 100 positions, indexed from 0 to 99.
We can use:

$$h(k) = k \bmod 10$$

Key	Calculation	Index
1023	1023 mod 100	23
1045	1045 mod 100	45
1087	1087 mod 100	87
1120	1120 mod 100	20

Can we map many possible keys to a small array? What would be the issue? What if we have more than 100 students?



A first Hash function

Suppose the array has 100 positions, indexed from 0 to 99.
We can use:

$$h(k) = k \bmod 10$$

Key	Calculation	Index
1023	1023 mod 100	23
1045	1045 mod 100	45
1087	1087 mod 100	87
1120	1120 mod 100	20

Can we map many possible keys to a small array? What would be the issue? What if we have more than 100 students? What if we have two students with 1023 and 2623 as IDs?



What Is a Hash Table?

A hash table stores key-value pairs in an array.

- A key identifies an entry.
- A value is the data associated with that key.
- A hash function helps determine where the entry should be stored.

$key \rightarrow hashfunction \rightarrow arrayindex \rightarrow value$



Properties of a useful Hash function



Properties of a useful Hash function

- Deterministic: the same key gives the same result.
- Efficient: calculating the result should be inexpensive.
- Valid index: the final index must be within the table.
- Good distribution: keys should spread reasonably across positions.



When two keys share an index

Use the same function:

$$h(k) = k \bmod 10$$

Key	Index
12	2
22	2
32	2

All three keys map to index 2.
This is a collision.



Can we avoid every Collision?

- There may be millions of possible keys.
- The array may have only a few hundred positions.
- Multiple keys must therefore map to the same position.

A better hash function can reduce collisions, but cannot eliminate them in general.



Two Main Strategies

Separate Chaining

Each array position stores a collection of entries.

Open Addressing

All entries stay inside the array. If a position is occupied, try another.



Strategy 1: Separate Chaining

Each position points to a list of entries.

```
Index
  0
  1
  2   ---> [12] ---> [22] ---> [32]
  3
  4
  ...
```

Keys 12, 22, and 32 all have hash index 2.



Inserting with Chaining

For each key-value pair:

1. Calculate the index.
2. Access the list.
3. Add the entry to that list.

```
int index = hash(key);
lists[index].add(new Entry(key, value));
```



Searching with Chaining

1. Calculate the index.
2. Visit entries in that list.
3. Compare keys until the matching entry is found.

```
int index = hash(key);
for (Entry entry : buckets[index]) {
    if (entry.key.equals(key)) {
        return entry.value;
    }
}
return null;
```



Strategy 2: Open Addressing

All entries are stored directly in the array.
If the preferred position is occupied, probe other positions until:

- an empty position is found, or
- the desired key is found.



Linear probing

Try consecutive positions after a collision.

$$h_i(k) = (h(k) + i) \bmod m$$

- $h(k)$: initial index
- i: probe number, starting at 0
- m: table size



Linear probing: Step by Step

Table size: 10

$$h(k) = k \bmod 10$$

Insert 12, 22, and 32.

1. 12 $\rightarrow$ index 2 (empty)
2. 22 $\rightarrow$ index 2 occupied; try index 3
3. 32 $\rightarrow$ index 2 occupied; try 3, then 4



Result of Linear Probing

Index:	0	1	2	3	4	5	6	7	8	9
Value:	-	-	12	22	32	-	-	-	-	-

Searching for 32:

- Check index 2: not 32.
- Check index 3: not 32.
- Check index 4: found 32.



Deletion with Linear Probing

Suppose the table contains 12, 22, and 32 at indices 2, 3, and 4.
If we simply clear index 3, searching for 32 might stop at that empty slot and incorrectly report that 32 is absent.
Solution: mark the slot as deleted (a tombstone), or use another carefully designed deletion strategy.



Chaining vs. Linear Probing

Property	Chaining	Linear probing
Storage	Bucket collections	Array only
Collisions	Add to same bucket	Search for another slot
Table can fill completely	Yes, with longer chains	No
Main concern	Long list	Clustering
Deletion	Remove from list	Must preserve probe sequence



Implementation plan

We will build a simplified hash table in stages:

1. Choose a table representation.
2. Calculate an index.
3. Insert a key-value pair.
4. Search for a key.
5. Handle collisions.
6. Delete entries.
7. Resize when needed.



A Simple Entry Class

```
class Entry {
    int key;
    String value;

    Entry(int key, String value) {
        this.key = key;
        this.value = value;
    }
}
```



Hash Function for Integer Keys

```
int hash(int key, int capacity) {
    return key % capacity;
}
```

- Produces an index from 0 to capacity - 1.
- Works with negative integers as well.



Implementing Chaining: Structure

```
class HashTable {
    private final List<Entry>[] buckets;

    HashTable(int capacity) {
        buckets = new List[capacity];
        for (int i = 0; i < capacity; i++) {
            buckets[i] = new ArrayList<>();
        }
    }
}
```



Implementing Chaining: Put

```
void put(int key, String value) {
    int index = hash(key, buckets.length);

    for (Entry entry : buckets[index]) {
        if (entry.key == key) {
            entry.value = value;
            return;
        }
    }

    buckets[index].add(new Entry(key, value));
}
```



Implementing Chaining: Get

```
String get(int key) {
    int index = hash(key, buckets.length);

    for (Entry entry : buckets[index]) {
        if (entry.key == key) {
            return entry.value;
        }
    }

    return null;
}
```



Test Collisions

```
HashTable grades = new HashTable(10);

grades.put(12, "A");
grades.put(22, "B");
grades.put(32, "C");

System.out.println(grades.get(12));
System.out.println(grades.get(22));
System.out.println(grades.get(32));
```

Expected output:

```
A
B
C
```



Implementation Exercise: Linear Probing

Ask students to implement a second version using arrays.
Required operations:

- put(key, value)
- get(key)
- remove(key)
- Handle collisions using linear probing.
- Detect when the table is full.



The Load Factor

The load factor measures how full a table is:

$$\alpha = \frac{n}{m}$$

- n: number of stored entries
- m: number of table positions

Example: 7 entries in a table of capacity 10 gives $\alpha = 0.7$.



Resizing and Rehashing

When the table becomes too full:

1. Allocate a larger array.
2. Recalculate positions for stored entries.
3. Insert entries into the new table.

Why recalculate positions?

$$h(k) = k \bmod m$$

Changing m can change the index.



Time Complexity

Operation	Average case	Worst case
Search	$O(1)$	$O(n)$
Insertion	$O(1)$	$O(n)$
Deletion	$O(1)$	$O(n)$

Average-case performance depends on good distribution and a suitable load factor.



What If the Key Is a String?

Examples:

- "student-1023"
- "sara@example.com"
- "computer-science"

A string hash function combines information from its characters to produce an integer hash value.



Java: hashCode() and equals()

Java objects can provide:

- `hashCode()` to produce an integer hash code.
- `equals()` to determine whether two objects represent equal keys.

Contract:
If `a.equals(b)` is true, then `a.hashCode()` = b.hashCode()= must also be true.
The reverse is not required.



A Common Mistake

If a custom class overrides `equals()` but not `hashCode()`, equal objects may produce different hash codes.
A hash-based collection may then fail to find an entry using an equal key.



Java HashMap

```
Map<Integer, String> grades = new HashMap<>();

grades.put(1023, "14");
grades.put(1045, "16");

System.out.println(grades.get(1023));
```

- Stores key-value pairs.
- Keys are unique.
- Provides efficient average-case lookup.
- Handles collisions and resizing internally.



Java HashSet

```
Set<String> names = new HashSet<>();

names.add("Ahmed");
names.add("Sara");
names.add("Ahmed");

System.out.println(names.size()); // 2
```

A HashSet stores unique elements and is useful for membership tests and duplicate detection.



What Other Languages Provide

- Python: `dict` and `set`
- JavaScript: `Map` and `Set`
- C++: `std::unordered_map` and `std::unordered_set`
- C#: `Dictionary<TKey, TValue>` and `HashSet<T>`



Where Are Hash Tables Useful?

- Find a student by ID.
- Count word frequencies.
- Detect duplicate values.
- Cache previously computed results.
- Track objects already processed.
- Index records by a key.



Example: Counting Word Frequencies

```
Map<String, Integer> counts = new HashMap<>();

for (String word : words) {
    counts.put(word, counts.getOrDefault(word, 0) + 1);
}
```

The word is the key; its frequency is the value.



When Is a Hash Table Not the Best Choice?

- When keys must remain sorted.
- When efficient range queries are required.
- When memory is very limited.
- When worst-case guarantees are essential and not provided by the chosen implementation.



Choosing a Map in Java

Type	Main characteristic
`HashMap`	General-purpose key-value lookup
`LinkedHashMap`	Predictable iteration order
`TreeMap`	Sorted keys
`ConcurrentHashMap`	Concurrent access support



The Central Idea

- Direct access is fast but requires suitable indices.
- Hashing maps keys to array positions.
- Collisions are inevitable in general.
- Chaining and open addressing resolve collisions differently.
- Load factor and resizing affect performance.



Key Takeaways

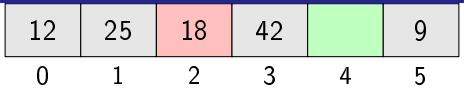
- Average-case lookup, insertion, and deletion can be $O(1)$.
- Worst-case operations can be $O(n)$.
- Hash quality and load factor matter.
- In Java, correct implementations of `equals()` and `hashCode()` are essential.
- Use standard collections in real applications; understand the implementation to use them well.



Questions

- Why are collisions unavoidable in general?
- How does chaining differ from linear probing?
- Why can deletion be tricky with open addressing?
- Why does resizing require rehashing?
- Why must equal Java objects have equal hash codes?




12
25
18
42
9
0
1
2
3
4
5